ADR 0109 — A conflict write names the repository it mutates

Status: Accepted — implemented and mutation-proved **Date:** 2026-09-03
Issue: #621 **Extends:** ADR 0003, ADR 0064, ADR 0069, and ADR 0103
Supersedes: the select-before-every-conflict-write mitigation in ADR 0105, decision 5 **Superseded by:** nothing

---

Context

Every conflict read can name an opaque worktree explicitly with `?repo=`. Neither conflict write could name one:

```
GET  /api/conflicts?repo=A
GET  /api/blob/{oid}?repo=A
GET  /api/worktree-file/{path}?repo=A
GET  /api/conflict-source/{path}?repo=A

POST /api/resolve-conflict
POST /api/resolve-conflict-content
```

The writes went through the shared planner, which resolved the session's selected repository. That makes this sequence possible:

1. a client explicitly inspects repository A;
2. the session selection is B because it was stale, was never set by this client, or another tab changed it;
3. A and B both have a conflict at `src/lib.rs`;
4. the client posts `TakeOurs` for `src/lib.rs`; and
5. every handler, planner and executor check freshly reads B, agrees that B is conflicted at that path, writes B, and reports success.

This is not a missing executor gate. Every gate is individually correct about the repository it was given. The mismatch is between a read request that named A and a write request that had no field in which to name A. No re-read of B can discover information the request never carried.

ADR 0069's `expected_stages` and `expected_source` narrow the exposure for content resolutions. They do not close it. Equal conflict stages and equal marker bytes can occur in two repositories, particularly related clones, and the tokens are not a repository selector. More importantly, whole-side `TakeOurs`, `TakeTheirs`, and `TakeDeletion` requests carry neither anchor.

ADR 0105 therefore made gv-tui pair every conflict write with its own `POST /api/select`. That was an explicit per-client mitigation pending #621. It reduced the window but made targeting depend on one request changing session navigation before a second request used that mutable state.

Decision

The contract decisions are closed as follows:

Question	Decision	Reason
Where does the selector travel?	Required `repo` field in each JSON body	Repository, path, and resolution are one mutation intent and one idempotent retry body.
What if `repo` disagrees with session selection?	Act on the named repository	Explicit request state outranks unrelated mutable navigation state.
Required or optional?	Required; protocol v12	An omitted field with selection fallback recreates the same successful wrong-repository write for every caller that omits it.
What happens to gv-tui's select pairing?	Remove it	Its cause is gone; retaining it adds a state mutation and race with no targeting value.

1. Both write bodies carry the required opaque worktree id

`ResolveConflictRequest` and `ResolveConflictContentRequest` each contain:

```
pub repo: String
```

`repo` has the same meaning as the conflict reads' query selector: it is the opaque `WorktreeId` registered in the server-owned catalog, never a filesystem path supplied by a client. A whole-side body is now shaped like:

```
{
  "repo": "<opaque-worktree-id>",
  "path": "src/lib.rs",
  "resolution": { "choice": "take_ours" }
}
```

The selector belongs in the body rather than the query because it is part of the write intent, beside the path and resolution that acquire one idempotency key. Keeping the target in that same strict, `deny_unknown_fields` DTO makes a captured request and a retry self-contained. The reads use a query because a GET addresses a resource without a body; that transport choice does not make a split query/body mutation preferable.

The field is not `Option<String>`. Old clients cannot be allowed through and then silently use selection fallback, so `PROTOCOL_VERSION`, `MIN_CLIENT_PROTOCOL`, and `MAX_CLIENT_PROTOCOL` move together from 11 to 12. A v11 client is refused during negotiation before it can send an unsafe old body.

Malformed ids return 400. Well-formed ids absent from the catalog return 404. The request can never supply a path for the server to trust.

2. The named repository is authoritative; selection is not a consistency check

When the body names A and session selection names B, the planner resolves and mutates A. It does not reject the disagreement and it does not select A first.

Refusing on disagreement would preserve the hidden dependency in another form. A second tab changing navigation to B would make a complete request for A fail for a fact unrelated to its intent, and every client would still need a select-before-write sequence. That sequence has a gap between the select and the write and changes session navigation as a side effect of resolving a file. Once the request carries an authoritative target, comparing it with mutable navigation adds failure modes but no safety.

The existing session Active/Visualize mode remains the permission gate. A Visualize-mode session still receives 403. Subject to that gate, the body says *where* the operation runs; the selection does not.

3. Explicit targeting joins the existing planner lifecycle

The two handlers parse `repo` and call `planner::plan_and_execute_for_worktree`. That entry resolves the id through the catalog, freshly revalidates the catalog path's gitdir and commondir geometry, and then feeds the resulting path and exact repository/worktree handle through the same build, validation, coordinator, execution, durable journal, and response lifecycle as selection-scoped writes. It never changes the session selection.

The target is also part of idempotency admission. `GitOperation` describes the conflict action but does not contain its repository, so comparing only the operation hash would let this sequence return the wrong success without running either request again:

```
key K + TakeOurs(path) + repo A -> success recorded for A
key K + TakeOurs(path) + repo B -> replay A's success as B's answer
```

Admission now requires the operation hash, repository token, and worktree token all to match. A changed operation or changed target receives 409. Both tokens matter: linked worktrees share a repository token while naming different working trees.

4. Each client carries the repository it inspected

The browser's conflict viewer takes the worktree id from the loaded `Frame` and passes it to every conflict read and both writes. That id is latched to the document being shown; a later session selection cannot retarget its controls.

gv-tui already carries `repo` in its conflict request enum. It now serializes that value directly into both write DTOs and sends one resolution POST. The `select_for_write` helper, the preparatory `/api/select`, and the test that required

the two-call ordering are removed. Its replacement test asserts both that the body names the repository and that no select is posted. A mitigation left after its cause is fixed would be cargo-cult state mutation, not defence in depth.

No other gv-tui file changes, and `route_authz.rs` is unchanged: methods, paths, authentication, CSRF, and LAN route presence did not change.

5. Content anchors stay, but they do their own job

Content resolutions still echo and re-check `expected_stages` and `expected_source` under the coordinator lock. They protect the document the user edited from changing within the named repository. `repo` protects the identity of that repository. Neither substitutes for the other, and whole-side resolutions now receive the repository identity they previously lacked entirely.

Alternatives considered

Add an optional body field

Rejected. Optionality is compatibility only by preserving the defect. Any client, script, or cached build that omits the field would still resolve the session selection and could still succeed against the wrong repository.

Put required `repo` in the query string

Rejected. It can technically close the hole, but splits one write intent between URL and body and makes idempotent request capture less self-contained. The query convention is appropriate for bodyless reads; it is not a reason to separate the mutation target from the mutation DTO.

Refuse whenever body `repo` and selection disagree

Rejected. Selection is navigation state, not an optimistic-lock token. A cross-tab navigation change would veto an otherwise complete request and force clients to retain the select-before-write dance. The explicit id is sufficient to resolve one fail-closed catalog target.

Select the named repository inside the handler, then use the old planner entry

Rejected. It would make conflict resolution change navigation, retain the race between selection and detached execution, and disguise an explicit-target contract as ambient state before the safety-critical layer reads it.

Rely on ADR 0069's content tokens

Rejected. Whole-side resolutions have no such token, and content identity can coincide across repositories. Tokens answer “is this still the content I edited?”;

they do not answer “which repository did I ask to edit?”

Consequences

Good: a conflict write cannot silently follow a stale or cross-tab selection; request logs and retries carry the target they mean; unknown ids fail closed through the catalog; both clients use one contract; the terminal loses one network round trip and one session side effect per write.

Costs: protocol v12 is intentionally incompatible with v11; every direct caller must add `repo`; the planner has a second composed-write entry; and idempotency admission now treats a target change as a key collision even when the `GitOperation` bytes are identical.

Decision log and acceptance evidence

The load-bearing regression is `a_whole_side_resolution_acts_on_the_named_repo_not_the_selection`. It creates two real repositories with a conflict at the identical path `a.txt`, explicitly inspects A, then makes B the Active session selection. It snapshots both B's three unmerged index stages and B's marker-file bytes, posts a real `TakeOurs` request naming A through the handler and planner, and asserts:

- the response succeeds;
- B's index is byte-for-byte unchanged;
- B's worktree file is byte-for-byte unchanged; and
- A has no unmerged index entry afterwards.

Before the fix, the test ran one test and failed on the B-index assertion: the left side was empty while the right side held stages 1, 2, and 3. A test that only proved `repo` deserialized would have stayed green on that implementation.

Additional direct checks:

Invariant	Evidence
Both DTOs require and round-trip `repo`	`conflict_write_bodies_require_and_round_trip_the_repository`; `a_conflict_write_cannot_omit_its_repository`
Both handlers enter the explicit-target planner	`every_git_write_route_reaches_the_planner`
A request naming A cannot write selected B	`a_whole_side_resolution_acts_on_the_named_repo_not_the_selection`
Idempotency binds separate repositories and linked worktrees	`the_same_key_and_operation_with_a_different_target_is_a_conflict`
gv-tui sends the target and no longer selects	`a_write_names_its_repository_without_selecting_it`; `a_content_resolution_posts_the_stages_and_token_it_was_handed`
Old clients stop at negotiation	`protocol_v12_is_a_hard_compatibility_window`

Mutation matrix

`failure-atlas mutation_check` ran from clean commit `f49bd53d`. Each run first executed an unmutated one-test baseline, then applied one exact edit in its contained clone. Both mutations compiled, reached the behavioral assertion, and emptied B's conflict index. Compiler failures are not counted.

Shape	Mutation	Baseline	Mutated leg	Result
Remove at the HTTP/planner handoff	Parse body `repo`, discard it, and call selection-scoped `plan_and_execute`	1 passed	0 passed, 1 failed at “B's conflict index changed”	**caught**, record 180
Weaken inside the planner boundary	Keep the handler correct but route `plan_and_execute_for_worktree` through `MutationTarget::Selection`	1 passed	0 passed, 1 failed at the same B-index oracle	**caught**, record 181

Total: **2/2 conclusive catches; 0 survived; 0 baseline or build failures**. The first attempted invocation, record 179, was correctly inconclusive: the atlas refused `/tmp` because this host has a `/tmp/.git` marker. It is excluded from the matrix. The conclusive runs used the atlas's validated `/var/tmp` base, and both report the source tree clean.

Verification

- `cargo test -p git-vista-protocol`: **230 passed, 0 failed, 1 ignored**.
- `cargo test -p gv-tui --bins`: **162 passed, 0 failed**.
- `cargo test -p git-vista`: **773 passed, 0 failed, 2 ignored** in the real `git-vista-ui` binary; the 0-test library target was reported separately and was not used as evidence.
- `cargo test -p git-vista-server -j 1`: **1,146 passed, 0 failed, 6 ignored** across the server binary, sandbox binary, and four integration targets.
- `cargo clippy --workspace --all-targets -- -D warnings`: passed.
- `cargo clippy -p git-vista --target wasm32-unknown-unknown --all-targets -- -D warnings`: passed.
- `trunk build`: passed and produced the real wasm distribution.
- `cargo fmt --all -- --check` and `git diff --check`: passed.
- failure-atlas: **2 caught, 0 survived** in conclusive records 180–181.

Signed: codex · 2026-09-03